

LLM Mastery Guide

Large Language Models: From Basics to Advanced

Tokenization • Architecture • Training • Fine-Tuning • Hallucination • Integration

A Beginner-to-Advanced Study Companion

Table of Contents

1. Introduction — What Is an LLM?
2. Core Building Blocks: Neural Networks Recap
3. Tokenization — Basics to Advanced
4. Embeddings & Positional Encoding
5. The Transformer Architecture
6. How LLMs Are Trained (Pretraining)
7. Fine-Tuning, RLHF, and Alignment
8. Parameter-Efficient Fine-Tuning (LoRA, QLoRA, PEFT)
9. Hallucination — Why It Happens & How to Reduce It
10. Evaluating LLMs
11. Adjusting & Customizing Tokenization
12. Integrating LLMs Into Applications
13. Prompt Engineering Essentials
14. Retrieval-Augmented Generation (RAG)
15. AI Agents & Tool Use
16. Glossary of Key Terms
17. Suggested Learning Path & Resources

1. Introduction — What Is an LLM?

A Large Language Model (LLM) is a neural network trained on massive amounts of text to predict the next piece of text (a **token**) given everything that came before it. That simple objective — 'predict the next token' — turns out to be powerful enough that, at large scale, models learn grammar, facts, reasoning patterns, coding ability, and conversational skill, all as a side effect of getting very good at prediction.

1.1 Why 'Large'?

'Large' refers to three things scaling together: the number of **parameters** (the model's learnable weights, often billions), the size of the **training data** (trillions of tokens scraped from books, code, websites, and more), and the amount of **compute** used to train it (measured in FLOPs, often costing millions of dollars in GPU time).

1.2 A Brief Timeline

Year	Milestone
2017	'Attention Is All You Need' introduces the Transformer architecture
2018	GPT-1 and BERT show transfer learning works well for language
2020	GPT-3 demonstrates strong few-shot learning at 175B parameters
2022	ChatGPT popularizes instruction-tuned, chat-style LLMs
2023-24	Open-weight models (Llama, Mistral) and multimodal LLMs emerge
2025-26	Long-context, agentic, and reasoning-focused models become standard

1.3 What an LLM Is NOT

- It is not a database — it does not 'look up' facts, it generates plausible text statistically.
- It does not truly 'understand' in the human sense — it models statistical relationships between tokens.
- It has no persistent memory between separate conversations unless a system is built around it to add one.

■ *Keep this mental model throughout the guide: an LLM is a very sophisticated next-token prediction machine. Almost every strength and weakness you'll learn about (including hallucination) traces back to this core fact.*

2. Core Building Blocks: Neural Networks Recap

Before tackling transformers, it helps to remember the basics of neural networks, since LLMs are built entirely from these components.

2.1 Neurons, Weights, and Layers

A neural network is a stack of layers. Each layer takes a vector of numbers as input, multiplies it by a matrix of learned **weights**, adds a **bias**, and passes the result through a non-linear **activation function** (like ReLU or GELU). Stacking many such layers lets the network approximate very complex functions.

2.2 Training via Gradient Descent

1. The model makes a prediction.
2. A **loss function** measures how wrong the prediction was (for LLMs, this is almost always **cross-entropy loss** on the next-token prediction task).
3. **Backpropagation** computes how much each weight contributed to the error.
4. An **optimizer** (commonly AdamW for LLMs) nudges every weight slightly in the direction that reduces the loss.
5. Repeat over billions of examples until the loss stops improving meaningfully.

2.3 Why Sequence Data Is Different

Language is a sequence: the meaning of a word depends on the words around it, sometimes far away in the text. Older architectures (RNNs, LSTMs) processed text one token at a time, which made them slow and forgetful over long sequences. The Transformer (Section 5) solves this by letting every token look at every other token directly, in parallel.

3. Tokenization — Basics to Advanced

Tokenization is the process of converting raw text into the discrete units (**tokens**) that a model actually consumes. It is one of the most underrated topics in LLMs — it quietly affects cost, context length, multilingual performance, and even arithmetic ability.

3.1 Why Not Just Use Words or Characters?

Approach	Pros	Cons
Character-level	Tiny vocabulary, no unknown-word problem	Very long sequences; hard to learn meaning per symbol
Word-level	Intuitive, short sequences	Huge vocabulary; can't handle new/rare words (typos, made-up words)
Subword-level	Balances vocabulary size and sequence length	Slightly less intuitive, doesn't always align with real words

Modern LLMs use **subword tokenization** — splitting text into pieces that are smaller than words but larger than single characters, so common words stay whole and rare words get split into meaningful chunks.

3.2 Byte-Pair Encoding (BPE) — The Workhorse Algorithm

BPE starts with individual characters (or bytes) and iteratively merges the most frequent adjacent pair into a new token, repeating thousands of times to build a vocabulary.

```
Start: l o w l o w e r n e w e s t
Step 1: merge 'e','s' -> 'es' (most frequent pair)
Step 2: merge 'es','t' -> 'est'
Step 3: merge 'l','o' -> 'lo'
...continue until target vocab size is reached
Result: 'lowest' -> ['low', 'est']
```

GPT-family models use a byte-level variant of BPE, which operates on raw UTF-8 bytes so it can represent literally any text (emoji, any language, binary data) without ever hitting an 'unknown token'.

3.3 Other Tokenization Algorithms

- **WordPiece** (used by BERT) — similar to BPE, but merges pairs that maximize the training data's likelihood rather than pure frequency.
- **Unigram Language Model** (used by SentencePiece/T5, ALBERT) — starts with a large vocabulary and prunes tokens that contribute least to the likelihood of the training corpus, rather than building up from scratch.
- **SentencePiece** — not an algorithm itself but a tokenizer *framework* that implements BPE or Unigram and treats the input as a raw stream (including spaces) so it works consistently across languages, even ones without spaces like Japanese or Thai.

3.4 Anatomy of a Tokenizer Vocabulary

- **Vocabulary size** — typically 30K to 250K tokens. GPT-4-class models use roughly 100K–200K.
- **Special tokens** — reserved tokens with specific meaning, e.g. `<|endoftext|>`, `[CLS]`, `[SEP]`, `<pad>`, `<s></s>`, and role markers like `<|user|>` / `<|assistant|>` used in chat templates.

- **Merge rules / vocab file** — the list of merges (BPE) or subword pieces (Unigram) learned from data, stored alongside the model so encoding/decoding stays consistent.

3.5 Tokenization Quirks Every Beginner Should Know

- **Numbers split unevenly** — '1234' might become ['123','4'] or ['12','34'] depending on the tokenizer, which is part of why LLMs sometimes struggle with arithmetic.
- **Leading spaces matter** — in BPE tokenizers like GPT's, ' hello' and 'hello' are often different tokens.
- **Non-English languages can cost more tokens** — a tokenizer trained mostly on English text will often split other languages (especially non-Latin scripts) into more, smaller tokens, making them more expensive and slower to generate.
- **Token count $\neq$ word count** — as a rough rule of thumb, 1 token $\approx$ 4 characters $\approx$ 0.75 words in English.

■ If an LLM ever seems to 'miscount letters' in a word (e.g. counting the letters in 'strawberry'), it's usually a tokenization effect — the model sees a few subword chunks, not individual letters, so letter-level tasks are genuinely harder for it.

4. Embeddings & Positional Encoding

4.1 Token Embeddings

Once text is tokenized into integer IDs, each ID is mapped to a dense vector (e.g. 4096 numbers) via an **embedding table** — essentially a big lookup matrix learned during training. Tokens with similar meanings end up with similar vectors after training.

4.2 Why Position Matters

The core attention mechanism (Section 5) has no built-in sense of word order — mathematically, it treats input as a set, not a sequence. **Positional encodings** are added to token embeddings so the model can tell 'dog bites man' apart from 'man bites dog'.

- **Sinusoidal positional encoding** (original Transformer) — fixed sine/cosine patterns added to embeddings.
- **Learned positional embeddings** (GPT-2 style) — a trainable vector per position, learned like any other weight.
- **Rotary Position Embedding (RoPE)** — used by most modern LLMs (Llama, GPT-NeoX-style); rotates query/key vectors by an angle proportional to position, which generalizes better to long or unseen sequence lengths.
- **ALiBi** — adds a distance-based penalty directly to attention scores instead of modifying embeddings.

5. The Transformer Architecture

The Transformer, introduced in 2017, is the architecture behind virtually every modern LLM. Its key idea is **self-attention**: every token directly computes how much it should 'pay attention to' every other token, in parallel, rather than processing the sequence step by step.

5.1 Self-Attention, Intuitively

For each token, the model creates three vectors: a **Query** (what am I looking for?), a **Key** (what do I contain?), and a **Value** (what information do I offer?). The attention score between two tokens is computed by comparing one token's Query to another's Key; higher similarity means more attention. The output for each token is a weighted blend of all Value vectors, weighted by these attention scores.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

5.2 Multi-Head Attention

Rather than computing attention once, the model does it several times in parallel ('heads'), each with its own learned Q/K/V projections. One head might learn to track grammatical subject-verb agreement, another might track coreference ('it' referring back to 'the cat'), and so on. The heads' outputs are concatenated and projected back to the model's dimension.

5.3 Decoder-Only Architecture (GPT-style)

Most modern LLMs are **decoder-only**: a stack of identical blocks, each containing masked self-attention (a token can only attend to itself and earlier tokens — this 'causal mask' is what makes autoregressive generation possible) followed by a feed-forward network, with residual connections and layer normalization around each.

```
Input tokens
-> Token Embedding + Positional Encoding
-> [Masked Self-Attention -> Add & Norm -> Feed-Forward -> Add & Norm] x N layers
-> Final Layer Norm
-> Linear projection to vocabulary size
-> Softmax -> probability distribution over next token
```

5.4 Mixture of Experts (MoE)

Some advanced models (e.g. Mixtral, some GPT-4-class and Gemini-class models) use **Mixture of Experts**: instead of one dense feed-forward network per layer, there are many 'expert' sub-networks, and a small router selects only a few experts per token. This allows a much larger total parameter count while keeping the compute cost per token roughly constant.

6. How LLMs Are Trained (Pretraining)

6.1 The Pretraining Objective

The vast majority of an LLM's capability comes from **pretraining**: training on huge unlabeled text corpora with a self-supervised objective — usually **next-token prediction** (also called causal language modeling). No human labels are needed; the 'label' for any position is simply the next token in the text itself.

6.2 Data

- Sources typically include filtered web crawls (e.g. Common Crawl), books, code repositories, encyclopedic content, academic papers, and licensed/curated datasets.
- **Data quality and deduplication** matter enormously — repeated or low-quality text can hurt performance and even be memorized verbatim.
- **Data mixture** (the ratio of code vs. web text vs. books, etc.) is tuned deliberately, since it strongly shapes downstream skills like reasoning and coding ability.

6.3 Scaling Laws

Research (notably the Kaplan et al. and Chinchilla papers) found predictable relationships between model size, dataset size, compute budget, and resulting loss. A key practical takeaway: for a fixed compute budget, there's an optimal balance between how large the model should be and how much data it should see — many earlier large models were actually undertrained relative to their size.

6.4 Training Mechanics

- **Batching & sequence packing** — many text sequences are packed into fixed-length training examples (e.g. 4096 or 8192 tokens) to use GPU memory efficiently.
- **Mixed precision training** (bf16/fp16) — speeds up training and reduces memory use.
- **Distributed training** — huge models are split across thousands of GPUs using data parallelism, tensor parallelism, and pipeline parallelism simultaneously.
- **Learning rate schedules** — typically a warmup phase followed by gradual decay (cosine or linear) over training.
- **Checkpointing & evaluation** — the model is periodically evaluated on held-out benchmarks throughout training to catch instabilities early.

■ *Pretraining a frontier-scale LLM from scratch costs millions of dollars and is only done by a handful of organizations. As a learner or developer, you'll almost always work with an already-pretrained model and adapt it (Sections 7-8) rather than training from zero.*

7. Fine-Tuning, RLHF, and Alignment

A raw pretrained model is good at completing text, but not necessarily at following instructions or being safe and helpful in conversation. Turning a 'base model' into an assistant involves further training stages.

7.1 Supervised Fine-Tuning (SFT)

The model is fine-tuned on a curated dataset of high-quality (prompt, ideal response) pairs, often written or reviewed by humans. This teaches the model the format and style of being a helpful assistant, including following instructions and using a consistent chat template.

7.2 Reinforcement Learning from Human Feedback (RLHF)

1. Humans (or AI evaluators) rank multiple model outputs for the same prompt from best to worst.
2. A separate **reward model** is trained to predict these human preference rankings.
3. The LLM is then fine-tuned using reinforcement learning (commonly **PPO**, or newer, simpler methods like **DPO** — Direct Preference Optimization) to produce outputs the reward model scores highly, while a penalty keeps it from drifting too far from the original SFT model.

7.3 Why Alignment Matters

Alignment training shapes the model's tendency to be helpful, honest, and harmless — including refusing harmful requests, avoiding excessive hedging, and calibrating confidence. It's an active research area, and the specific techniques (RLHF, DPO, Constitutional AI-style self-critique, RLAIIF) continue to evolve.

7.4 Full Fine-Tuning vs. Lightweight Adaptation

Method	What Changes	Typical Use Case
Full fine-tuning	All model weights updated	You have lots of compute and a large, high-quality dataset
LoRA / PEFT (Sec. 8)	A small number of added parameters	Most practical fine-tuning — cheap, fast, reversible
Prompt-based adaptation	No weights changed at all	Quick experiments, in-context learning, few-shot prompting

8. Parameter-Efficient Fine-Tuning (LoRA, QLoRA, PEFT)

Full fine-tuning of a multi-billion parameter model requires enormous GPU memory. **Parameter-Efficient Fine-Tuning (PEFT)** methods adapt a model by training only a small number of new parameters while freezing the original weights.

8.1 LoRA (Low-Rank Adaptation)

LoRA freezes the original weight matrices and injects small, trainable 'low-rank' matrices alongside them. Instead of learning a full update ΔW to a weight matrix W , LoRA learns ΔW as the product of two much smaller matrices A and B , drastically cutting the number of trainable parameters (often by 100-1000x) while retaining most of the quality gain from fine-tuning.

```
W_effective = W_frozen + (alpha / r) * (B @ A)
# W_frozen: original pretrained weights (untouched)
# A, B: small trainable matrices, rank r (e.g. r = 8, 16, 64)
# alpha: scaling factor
```

8.2 QLoRA

QLoRA combines LoRA with **quantization** — the frozen base model is loaded in 4-bit precision to save memory, while the small LoRA adapter matrices are still trained in higher precision. This makes it possible to fine-tune a 30-70B parameter model on a single high-end consumer or prosumer GPU.

8.3 Other PEFT Techniques

- **Adapters** — small extra layers inserted between existing transformer layers.
- **Prefix / Prompt Tuning** — a small set of trainable 'virtual tokens' prepended to the input, with all model weights frozen.
- **IA3** — learns per-channel scaling vectors instead of full weight updates, even lighter than LoRA.

■ *For most beginners and small teams wanting to customize a model on their own data, LoRA (or QLoRA if GPU memory is limited) is the standard, well-supported starting point via libraries like Hugging Face's PEFT.*

9. Hallucination — Why It Happens & How to Reduce It

Hallucination refers to an LLM generating text that is fluent and confident-sounding but factually incorrect, unsupported, or entirely fabricated (e.g. inventing a citation, a statistic, or an API function that doesn't exist).

9.1 Root Causes

- **The objective itself** — the model is trained to produce plausible, fluent continuations, not necessarily verified-true ones. Fluency and truth are correlated but not identical.
- **Knowledge cutoff & gaps** — the model has no information about events after its training data ends, or about very obscure/private topics, but is not always aware of the boundary of its own knowledge.
- **Compression and interpolation** — the model stores an approximate, compressed version of patterns in its training data, so it can 'blend' real facts into plausible-sounding but false combinations.
- **Exposure bias / long generations** — small early errors can compound over a long response, since each new token is generated conditioned on the model's own possibly-flawed earlier output.
- **Sycophancy** — models fine-tuned on human preference data can learn to agree with a user's framing even when it's wrong, since agreeable answers were often rated more highly by human labelers.
- **Ambiguous or leading prompts** — vague questions or ones that presuppose a false premise ('When did Einstein visit the Moon?') can push the model to fabricate an answer that fits the premise.

9.2 Types of Hallucination

Type	Example
Factual	Wrong dates, names, statistics stated confidently
Fabricated sources	Inventing a paper title, URL, or legal case that doesn't exist
Faithfulness / grounding	Misrepresenting a document the model was actually given to summarize
Logical / reasoning	A confidently wrong step in multi-step reasoning or math
Code hallucination	Calling a library function or parameter that was never real

9.3 Mitigation Techniques

- **Retrieval-Augmented Generation (RAG)** — ground answers in retrieved, verifiable documents rather than relying purely on parametric memory (see Section 14).
- **Lower temperature / careful decoding** — reduces randomness for factual tasks (see 9.4).
- **Ask for citations and check them** — explicitly instruct the model to cite sources, and verify those sources independently rather than trusting them blindly.
- **Chain-of-thought and self-consistency** — asking the model to reason step by step, and/or sampling multiple reasoning paths and taking a majority vote, reduces certain reasoning errors.

- **Encourage calibrated uncertainty** — prompt or fine-tune the model to say 'I'm not sure' rather than guessing, and reward that behavior during alignment training.
- **Fact-checking / verifier pipelines** — a second pass (another model call, a search tool, or a human) cross-checks claims before they're finalized.
- **Constrain the task** — narrow, well-specified prompts with relevant context supplied up front hallucinate far less than open-ended 'tell me everything about X' prompts.

9.4 Decoding Parameters and Hallucination

Parameter	Effect
Temperature	Controls randomness. Lower (e.g. 0-0.3) = more deterministic/factual; higher (0.7-1.0+) = more creative/varied.
Top-p (nucleus sampling)	Samples only from the smallest set of tokens whose cumulative probability exceeds p, cutting off unlikely (often
Top-k	Restricts sampling to the k most likely next tokens
Repetition penalty	Discourages repeating the same tokens/phrases, useful in long generations

■ No technique eliminates hallucination completely — it is a fundamental property of how these models generate text. The realistic goal is minimizing and catching it, not assuming it's fully solved.

10. Evaluating LLMs

10.1 Automatic Benchmarks

- **MMLU** — broad multiple-choice knowledge test across 57 subjects.
- **HumanEval / MBPP** — code generation correctness benchmarks.
- **GSM8K / MATH** — grade-school and competition math word problems, testing reasoning.
- **HellaSwag / ARC** — commonsense reasoning and science question benchmarks.
- **TruthfulQA** — specifically tests resistance to generating popular misconceptions.

10.2 Human & Model-Based Evaluation

- **Human preference comparisons** (e.g. Chatbot Arena-style A/B voting) capture real-world usefulness that static benchmarks miss.
- **LLM-as-a-judge** — using a strong model to score or compare outputs at scale, which is cheaper than human review but can inherit the judge model's own biases.

10.3 Practical Evaluation for Your Own Application

Public benchmarks rarely reflect your specific use case. When integrating an LLM into a product, build a small custom evaluation set of real examples from your domain, define clear pass/fail or scoring criteria, and re-run it whenever you change prompts, models, or fine-tuning.

11. Adjusting & Customizing Tokenization

'Adjusting the tokenizer' can mean a few different things depending on your goal. Here's how to think about each.

11.1 Training a New Tokenizer From Scratch

If you're pretraining a model for a new domain or language, you typically train a fresh BPE/Unigram tokenizer on a representative corpus using a library like Hugging Face tokenizers or Google's sentencepiece.

```
from tokenizers import ByteLevelBPETokenizer

tokenizer = ByteLevelBPETokenizer()
tokenizer.train(
    files=["corpus.txt"],
    vocab_size=32000,
    min_frequency=2,
    special_tokens=["<s>", "<pad>", "</s>", "<unk>", "<mask>"]
)
tokenizer.save_model("my_tokenizer")
```

11.2 Choosing Vocabulary Size — The Trade-off

Smaller vocab (e.g. 8K-32K)	Larger vocab (e.g. 100K-250K)
Longer token sequences for the same text	Shorter sequences -> cheaper & faster inference
Smaller embedding table (less memory)	Larger embedding table (more memory & compute per step)
Better for narrow, low-resource domains/languages	Better for broad, multilingual coverage

11.3 Extending an Existing Tokenizer (Domain Adaptation)

Rather than training from scratch, you can add new tokens to an existing pretrained model's tokenizer — useful for domain-specific jargon (medical, legal, code symbols) that currently gets split awkwardly.

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("base-model-name")
new_tokens = ["myoclonus", "tokenamide", "<CUSTOM_TAG>"]
num_added = tokenizer.add_tokens(new_tokens)

model = AutoModelForCausalLM.from_pretrained("base-model-name")
model.resize_token_embeddings(len(tokenizer))
# New token embeddings start randomly initialized -
# the model needs some fine-tuning to learn them well.
```

■ Adding tokens changes the embedding matrix size, so the model needs at least some fine-tuning afterward — the new tokens start with random (or averaged) embeddings and have no learned meaning yet.

11.4 Tuning Tokenization Behavior at Inference Time

- Some frameworks let you bias or ban specific tokens (**logit bias**) — useful for forcing or forbidding certain words/formats.
- **Special/control tokens** in chat templates (e.g. role markers) must exactly match what the model was fine-tuned with, or output quality degrades sharply — always use the official chat template for a given model.
- For multilingual or code-heavy use cases, check a model's **tokenizer efficiency** (tokens-per-word) on your actual target language/domain before committing — it directly affects cost and effective context window.

12. Integrating LLMs Into Applications

12.1 Common Integration Patterns

Pattern	Description
Direct API calls	Call a hosted model's API (e.g. Anthropic, OpenAI) from your backend
Self-hosted inference	Run an open-weight model yourself (vLLM, TGI, Ollama) for cost/privacy control
RAG pipeline	Retrieve relevant documents, then feed them to the LLM as context (Section 14)
Agent / tool use	LLM decides when to call external functions/APIs to accomplish a task (Section 15)
Fine-tuned model	Adapt a model to your data/style before deploying it (Sections 7-8)

12.2 A Minimal API Integration Example

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_KEY")

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1000,
    system="You are a concise, factual assistant.",
    messages=[
        {"role": "user", "content": "Summarize the plot of Hamlet in 3 sentences."}
    ]
)
print(response.content[0].text)
```

12.3 Key Design Considerations

- **Context window management** — track token usage so you don't exceed the model's max context; trim, summarize, or use RAG for long histories.
- **Streaming** — stream tokens to the user as they're generated for a responsive feel, instead of waiting for the full response.
- **Latency vs. cost vs. quality** — smaller/faster models for simple tasks, larger models reserved for complex reasoning, often via a routing layer.
- **Structured output** — use JSON mode / function-calling schemas when your app needs to parse the model's output programmatically, rather than parsing free text.
- **Guardrails** — input/output filtering, content moderation, and validation layers around the raw model call, especially for user-facing products.
- **Observability** — log prompts, responses, latency, and cost so you can debug and improve the system over time.

13. Prompt Engineering Essentials

Prompting is the primary way most people and applications 'program' an LLM without training it further.

13.1 Core Techniques

- **Be specific and give context** — vague prompts get vague, more hallucination-prone answers.
- **Few-shot examples** — show 1-5 examples of the input/output pattern you want directly in the prompt.
- **Chain-of-thought prompting** — ask the model to reason step by step before giving a final answer, which improves performance on multi-step problems.
- **Role/system prompts** — set persistent behavior, tone, and constraints via a system message.
- **Explicit output format** — specify the exact structure you want (JSON schema, bullet list, table) to make output easy to parse and more consistent.
- **Positive framing** — telling the model what *to* do is usually more reliable than only listing what not to do.

13.2 Iterating Like an Engineer

Treat prompts as code: version them, test them against a fixed set of examples, and change one variable at a time so you can tell what actually improved (or hurt) the result.

14. Retrieval-Augmented Generation (RAG)

RAG combines an LLM with an external knowledge source, retrieving relevant text at query time and inserting it into the prompt — grounding responses in real, up-to-date, verifiable information rather than only the model's frozen training data.

14.1 The Basic Pipeline

1. **Chunk** your documents into manageable pieces (e.g. 200-500 tokens each).
2. **Embed** each chunk into a vector using an embedding model, and store vectors in a vector database (e.g. Pinecone, Weaviate, pgvector, FAISS).
3. At query time, **embed the user's question** the same way, and retrieve the most similar chunks (nearest-neighbor search).
4. **Insert** the retrieved chunks into the prompt as context, then ask the LLM to answer using only that context.
5. Optionally **re-rank** retrieved chunks with a smaller, more precise model before the final generation step, to improve relevance.

14.2 Why RAG Helps with Hallucination

Instead of relying purely on facts memorized (and possibly distorted) during pretraining, the model is asked to synthesize an answer from text it can literally see in its context window, which is far more reliable and lets you cite exact sources.

15. AI Agents & Tool Use

An 'agent' pattern gives the LLM the ability to call external tools/functions (web search, calculators, code execution, databases, APIs) and use their results to complete multi-step tasks, rather than answering from its own knowledge alone.

15.1 The Agent Loop

1. The model receives a task and a list of available tools (with descriptions of what each does).
2. The model decides whether to answer directly or call a tool, and with what arguments.
3. The tool executes and returns a result to the model.
4. The model incorporates that result and either calls another tool or produces a final answer.

15.2 Practical Notes

- Tool descriptions should be clear and specific — the model chooses tools based on these descriptions, much like choosing which token to generate next.
- Always validate and sandbox tool calls that can take real-world actions (sending emails, making purchases, modifying files) — never let a model execute irreversible actions without appropriate checks.
- Multi-step agents can compound small errors, so adding verification steps or human review checkpoints for high-stakes actions is good practice.

16. Glossary of Key Terms

Term	Definition
Token	A unit of text (word, subword, or character) the model reads/generates.
Context window	The maximum number of tokens a model can consider at once (input + output).
Parameter	A single learnable weight in the model; large models have billions.
Autoregressive	Generating output one token at a time, each conditioned on all previous tokens.
Logits	Raw, unnormalized scores the model outputs for each possible next token, before softmax.
Softmax	A function converting logits into a probability distribution that sums to 1.
Fine-tuning	Further training a pretrained model on a smaller, task/domain-specific dataset.
Inference	Running a trained model to generate output (as opposed to training it).
Quantization	Reducing numeric precision of weights (e.g. 16-bit to 4-bit) to save memory/speed.
Context length extension	Techniques (e.g. RoPE scaling) allowing a model to handle longer inputs than it was originally trained on.
Prompt injection	A security risk where malicious text in input tricks a model into ignoring its instructions.
Grounding	Basing a model's output on specific, verifiable source material rather than memory alone.

17. Suggested Learning Path & Resources

17.1 A Beginner-Friendly Path

1. Get comfortable with Python and basic linear algebra/probability (vectors, matrices, dot products).
2. Understand classic neural networks and gradient descent (build a tiny one from scratch).
3. Read 'Attention Is All You Need' and, alongside it, a step-by-step Transformer walkthrough/implementation.
4. Play hands-on with a tokenizer (encode/decode text yourself) to build intuition from Section 3.
5. Use a hosted LLM API to build a small project (e.g. a Q&A; bot) — this cements Sections 12-13.
6. Learn RAG by building a simple retrieval pipeline over your own documents (Section 14).
7. Try parameter-efficient fine-tuning (LoRA) on a small open model to see training mechanics firsthand.
8. Study hallucination and evaluation deeply once you're building anything user-facing (Sections 9-10).
9. Explore agents/tool-use once comfortable with the basics (Section 15).

17.2 Types of Resources to Seek Out

- Original research papers for concepts you want to go deep on (Transformer, BPE, LoRA, RLHF, Chinchilla scaling laws — search these titles directly).
- Official documentation for whichever model provider or open-weight ecosystem you're using — implementation details and best practices change frequently.
- Open-source codebases of small models (e.g. nanoGPT-style projects) to read and run end-to-end training yourself at a toy scale.
- Hands-on notebooks/tutorials for the Hugging Face transformers, tokenizers, and peft libraries.

Final note: This field moves fast. Treat this guide as a map of the concepts and vocabulary you need — then go verify current best practices and the latest techniques against up-to-date documentation and papers, since specific numbers (model sizes, benchmark scores, vocab sizes) shift every few months.